



```
int r(int x)
{
    return r(x - 1);
}

int f(int a, int b, int c)
{
    if (c == 1) return a; else return b;
}
```

Figure 1: Esempio di pseudocodice

Rispondere alle domande a risposta multipla annerendo la casella corrispondente alla risposta corretta. Ogni domanda ha una ed una sola risposta corretta.

Cognome e Nome:

Matricola:

Domanda 1 In presenza di variabili modificabili:

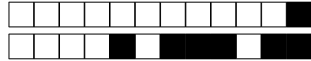
- ☐ La valutazione del comando di assegnamento restituisce sempre un valore
- ☐ Esistono un *Ambiente* che associa valori denotabili (fra cui le locazioni di memoria) a nomi ed una *Memoria* che associa locazioni di memoria a valori memorizzabili
- ☐ Nessuna delle altre risposte
- ☐ Non esistono valori denotabili
- ☐ Il comando di assegnamento non ha effetti collaterali

Domanda 2 β -riducendo $(\lambda a.((a\lambda b.\lambda c.c)\lambda d.\lambda e.d))(\lambda f.\lambda g.f)$ si ottiene:

- ☐ $\lambda b.\lambda c.c$
- ☐ Nessuna delle altre risposte
- ☐ La riduzione non termina
- ☐ c
- ☐ $\lambda b.\lambda c.b$

Domanda 3 Si consideri lo pseudo-codice di Figura 1. Qual'è il valore di ritorno di $f(1, r(1), 1)$ se i parametri sono passati *per nome*?

- ☐ Non è possibile dirlo senza conoscere il tipo di scope (statico o dinamico) utilizzato
- ☐ Nessuna delle altre risposte
- ☐ 1
- ☐ Si ha ricorsione infinita
- ☐ 0



```
int a, b, c;

void pippo(void)
{
    int a;

    a = 6;
    b = 5;
}

void pluto(void)
{
    int c;
    int b;

    pippo();
    c = 3;
    a = 4;
}

void topolino(void)
{
    int a;

    a = 1;
    b = 10;
    pluto();

    c = a + b;
}
```

Figure 2: Esempio di pseudocodice

Domanda 4 Dato il frammento di programma (espresso in pseudo-codice) della Figura 2, quanto vale la variabile globale `c` dopo aver eseguito `topolino()`, assumendo scope dinamico?

- ☐ 14
- ☐ 3
- ☐ Non è possibile dirlo
- ☐ 6
- ☐ Nessuna delle altre risposte

Domanda 5 I puntatori di catena dinamica contenuti in un record di attivazione:

- ☐ Nessuna delle altre risposte
- ☐ Non esistono “puntatori di catena dinamica” in un record di attivazione
- ☐ Servono per identificare la zona di memoria in cui è memorizzata una variabile locale
- ☐ Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa
- ☐ Servono per accedere alle variabili dinamiche



Domanda 6 β -riducendo $(\lambda x.xy)(\lambda z.zx)(\lambda z.zx)$ si ottiene:

- ☐ Nessuna delle altre risposte
- ☐ xyz
- ☐ $(\lambda x.xy)yx$
- ☐ $yx(\lambda z.zx)$
- ☐ La riduzione non termina

Domanda 7 Un *compilatore* da un linguaggio $\mathcal{L}$ ad un linguaggio $\mathcal{L}_O$ è:

- ☐ Nessuna delle altre risposte
- ☐ Una implementazione di macchine astratte indipendente dalla macchina fisica
- ☐ Un programma che trasforma un programma $P^{\mathcal{L}_O}$ (espresso nel linguaggio $\mathcal{L}_O$) in un programma $P^{\mathcal{L}}$ (espresso nel linguaggio $\mathcal{L}$) tale che per ogni input I si ha $P^{\mathcal{L}}(I) = P^{\mathcal{L}_O}(I)$
- ☐ L'implementazione di una macchina astratta scritta nel linguaggio $\mathcal{L}_O$, che capisce programmi scritti nel linguaggio $\mathcal{L}$
- ☐ Un programma scritto nel linguaggio $\mathcal{L}_O$ che riceve come ingresso un programma $P^{\mathcal{L}}$ (espresso nel linguaggio $\mathcal{L}$) ed il suo input I generando lo stesso output che genera $P^{\mathcal{L}}$ con input I

Domanda 8 I dangling pointer:

- ☐ Sono identificabili tramite tecniche di reference counting (contatore dei riferimenti)
- ☐ Sono un problema solo per il linguaggio Java
- ☐ Sono identificabili tramite la tecnica detta “mark and sweep”
- ☐ Non possono essere identificati con certezza, ma questo non è un problema perché comportano solo un piccolo spreco di memoria
- ☐ Nessuna delle altre risposte

Domanda 9 Dato il frammento di programma (espresso in pseudo-codice) contenuto in Figura 3, qual'è il valore di ritorno di `f1()`, assumendo scope statico?

- ☐ 5
- ☐ Nessuna delle altre risposte
- ☐ Non è possibile dirlo
- ☐ 0
- ☐ -5

Domanda 10 La *frammentazione esterna* causa:

- ☐ Uno spreco di memoria
- ☐ Nessuna delle altre risposte
- ☐ L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente
- ☐ Un rallentamento rilevante nelle operazioni di allocazione della memoria
- ☐ Il funzionamento non corretto di programmi che allocano memoria dinamicamente



```
int x, y, z;

void f3(void)
{
    x = 0;
    y = 5;
}

void f2(void)
{
    int y;

    f3();
    y = 0;
    z = 10;
}

int f1(void)
{
    int x;

    x = -5;
    y = 10;
    z = x + y;
    f2();

    return z - y - x;
}
```

Figure 3: Esempio di pseudocodice

Domanda 11 La *valutazione con corto circuito* del predicato “A && B” (dove “&&” rappresenta un and logico):

- ☐ Stabilisce che se “B” è vero allora “A” non viene valutato
- ☐ Nessuna delle altre risposte
- ☐ Stabilisce che se “A” è falso allora “B” non viene valutato
- ☐ Crea un non-determinismo nell’ordine della valutazione di “A” e “B”
- ☐ Stabilisce che “A” e “B” devono essere valutati in parallelo

Domanda 12 Se l’ambiente di una funzione non contiene il nome della funzione stessa:

- ☐ Nessuna delle altre risposte
- ☐ Non è possibile per la funzione invocarsi ricorsivamente
- ☐ Non ci sono particolari conseguenze
- ☐ La funzione non può usare scope dinamico
- ☐ La funzione non può usare scope statico



Rispondere alle domande a risposta multipla annerendo la casella corrispondente alla risposta corretta. Ogni domanda ha una ed una sola risposta corretta.

Cognome e Nome:

Matricola:

Domanda 1 La ricorsione in coda:

- ☐ Permette di risolvere il problema della ricorsione infinita
- ☐ Richiede di non scrivere mai la chiamata ricorsiva come ultimo statement di una subroutine
- ☐ Richiede di non ritornare mai direttamente il valore ritornato da una chiamata ricorsiva
- ☐ Nessuna delle altre risposte
- ☐ Permette di evitare un'eccessiva crescita della dimensione dello stack

Domanda 2 Dato il frammento di programma (espresso in pseudo-codice) contenuto in Figura 1, quanto vale la variabile globale `c` dopo aver eseguito `topolino()`, assumendo scope statico?

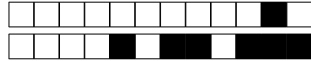
- ☐ 14
- ☐ Nessuna delle altre risposte
- ☐ 6
- ☐ 3
- ☐ Non è possibile dirlo

Domanda 3 Dato il frammento di programma (espresso in pseudo-codice) contenuto in Figura 2, qual'è il valore di ritorno di `topolino()`, assumendo scope statico?

- ☐ Nessuna delle altre risposte
- ☐ 0
- ☐ Non è possibile dirlo
- ☐ -3
- ☐ 11

Domanda 4 Se gli array sono memorizzati *per colonne* e `char a[100][100]` è un array multidimensionale di caratteri con `a[0][0]` che ha indirizzo `0x1100`, qual'è l'indirizzo di `a[5][10]`?:

- ☐ Nessuna delle altre risposte
- ☐ 0x24ED
- ☐ 0x21FE
- ☐ 0x22FE
- ☐ 0x14ED



```
int a, b, c;

void pippo(void)
{
    int a;

    a = 6;
    b = 5;
}

void pluto(void)
{
    int c;
    int b;

    pippo();
    c = 3;
    a = 4;
}

void topolino(void)
{
    int a;

    a = 1;
    b = 10;
    pluto();

    c = a + b;
}
```

Figure 1: Esempio di pseudocodice

```
int x, y;

void pippo(void)
{
    x = 8;
    y = 4;
}

void pluto(void)
{
    int y;

    pippo();
    y = 3;
}

int topolino(void)
{
    int x, z;

    x = 5;
    y = 15;
    z = x + y;
    pluto();

    return z - y - x;
}
```

Figure 2: Esempio di pseudocodice



Domanda 5 Un *interprete* di un linguaggio $\mathcal{L}$ scritto in un linguaggio $\mathcal{L}_O$ è:

- ☐ Nessuna delle altre risposte
- ☐ L'implementazione di una macchina astratta scritta nel linguaggio $\mathcal{L}_O$, che capisce programmi scritti nel linguaggio $\mathcal{L}$
- ☐ Una implementazione di macchine astratte indipendente dalla macchina fisica
- ☐ Un programma scritto nel linguaggio $\mathcal{L}_O$ che riceve come ingresso un programma $P^{\mathcal{L}}$ (espresso nel linguaggio $\mathcal{L}$) ed il suo input I generando lo stesso output che genera $P^{\mathcal{L}}$ con input I
- ☐ Un programma che trasforma un programma $P^{\mathcal{L}}$ (espresso nel linguaggio $\mathcal{L}$) in un programma $P^{\mathcal{L}_O}$ (espresso nel linguaggio $\mathcal{L}_O$) tale che per ogni input I si ha $P^{\mathcal{L}}(I) = P^{\mathcal{L}_O}(I)$

Domanda 6 β -riducendo $(\lambda a.aab)((\lambda a.aab)(\lambda a.(\lambda b.ba)c))$ si ottiene:

- ☐ La riduzione non termina
- ☐ $ccb(ccb)c$
- ☐ aab
- ☐ $\lambda a.aab$
- ☐ Nessuna delle altre risposte

Domanda 7 La *frammentazione esterna* causa:

- ☐ Il funzionamento non corretto di programmi che allocano memoria dinamicamente
- ☐ Un rallentamento rilevante nelle operazioni di allocazione della memoria
- ☐ Nessuna delle altre risposte
- ☐ L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente
- ☐ Uno spreco di memoria

Domanda 8 Il concetto di *variabile modificabile*:

- ☐ E' tipico del paradigma di programmazione imperativo
- ☐ Permette di evitare il fenomeno dell'*aliasing*
- ☐ Nessuna delle altre risposte
- ☐ E' l'unico concetto utilizzabile quando si parla di variabili
- ☐ E' imposto dall'architettura di Von Neumann (variabili non modificabili richiederebbero macchine astratte caratterizzate da memoria a sola lettura)

Domanda 9 Si consideri lo pseudo-codice di Figura 3. Qual'è il valore di ritorno di $f(1, r(1), 1)$ se i parametri sono passati *per nome*?

- ☐ Nessuna delle altre risposte
- ☐ Non è possibile dirlo senza conoscere il tipo di scope (statico o dinamico) utilizzato
- ☐ Si ha ricorsione infinita
- ☐ 1
- ☐ 0



```
int r(int x)
{
    return r(x - 1);
}

int f(int a, int b, int c)
{
    if (c == 1) return a; else return b;
}
```

Figure 3: Esempio di pseudocodice

```
int c = 2;

int pippo(int a)
{
    c = c + 2;
    return a * 2;
}

int pluto(void)
{
    return(pippo(c + 1));
}
```

Figure 4: Esempio di pseudocodice

Domanda 10 Si consideri lo pseudo-codice di Figura 4. Qual'è il valore di ritorno di `pluto()` se i parametri sono passati *per riferimento*?

- ☐ Non è possibile passare `c + 1` per riferimento
- ☐ Dipende dal tipo di scope (statico o dinamico) utilizzato
- ☐ 6
- ☐ Nessuna delle altre risposte
- ☐ 10

Domanda 11 La *frammentazione interna* causa:

- ☐ Il funzionamento non corretto di programmi che allocano memoria dinamicamente
- ☐ Un rallentamento rilevante nelle operazioni di allocazione della memoria
- ☐ Nessuna delle altre risposte
- ☐ Uno spreco di memoria
- ☐ L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente

Domanda 12 β -riducendo $(\lambda a.aaa)((\lambda b.b)(\lambda c.c))$ si ottiene:

- ☐ Nessuna delle altre risposte
- ☐ `aaa`
- ☐ La riduzione non termina
- ☐ `$\lambda x.xa$`
- ☐ `$\lambda a.a$`



Rispondere alle domande a risposta multipla annerendo la casella corrispondente alla risposta corretta. Ogni domanda ha una ed una sola risposta corretta.

Cognome e Nome:

Matricola:

Domanda 1 Se gli array sono memorizzati *per colonne* ed `short int a[100][100]` è un array multidimensionale di interi corti (si assuma che la dimensione di uno `short int` sia 2 byte) con `a[0][0]` che ha indirizzo `0x4100`, qual'è l'indirizzo di `a[5][10]`?:

- ☐ `0x48DA`
- ☐ `0x4510`
- ☐ Nessuna delle altre risposte
- ☐ `0x47DA`
- ☐ `0x41FE`

Domanda 2 La *frammentazione interna* causa:

- ☐ Il funzionamento non corretto di programmi che allocano memoria dinamicamente
- ☐ Nessuna delle altre risposte
- ☐ Un rallentamento rilevante nelle operazioni di allocazione della memoria
- ☐ Uno spreco di memoria
- ☐ L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente

Domanda 3 L'allocazione dinamica della memoria:

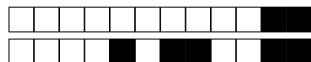
- ☐ Può essere fatta sia dallo *stack* che dallo *heap*
- ☐ Può essere fatta solo dallo *heap*
- ☐ Può essere fatta solo dallo *stack*
- ☐ Nessuna delle altre risposte
- ☐ E' sempre effettuata solo dal compilatore o dall'interprete

Domanda 4 Se l'ambiente di una funzione non contiene il nome della funzione stessa:

- ☐ Nessuna delle altre risposte
- ☐ La funzione non può usare scope dinamico
- ☐ La funzione non può usare scope statico
- ☐ Non è possibile per la funzione invocarsi ricorsivamente
- ☐ Non ci sono particolari conseguenze

Domanda 5 β -riducendo $((\lambda a.aaa)(\lambda b.b))(\lambda c.c)$ si ottiene:

- ☐ La riduzione non termina
- ☐ `aaa`
- ☐ `$\lambda x.xa$`
- ☐ Nessuna delle altre risposte
- ☐ `$\lambda c.c$`



Domanda 6 L'*ambiente* (o *environment*) è:

- ☐ L'insieme delle associazioni (nome, entità denotabile) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma
- ☐ L'insieme dei valori che una variabile assume durante l'esecuzione di un programma
- ☐ Un insieme di associazioni (nome, valore) definite staticamente durante lo sviluppo di un programma
- ☐ Nessuna delle altre risposte
- ☐ Una lista di coppie (nome, tipo) che permette di accedere alle variabili di un programma

Domanda 7 β -riducendo $(\lambda a.aaa)((\lambda b.b)(\lambda c.c))$ si ottiene:

- ☐ $\lambda a.a$
- ☐ Nessuna delle altre risposte
- ☐ aaa
- ☐ La riduzione non termina
- ☐ $\lambda x.xa$

Domanda 8 La *valutazione con corto circuito* del predicato "A && B" (dove "&&" rappresenta un and logico):

- ☐ Crea un non-determinismo nell'ordine della valutazione di "A" e "B"
- ☐ Nessuna delle altre risposte
- ☐ Stabilisce che "A" e "B" devono essere valutati in parallelo
- ☐ Stabilisce che se "A" è falso allora "B" non viene valutato
- ☐ Stabilisce che se "B" è vero allora "A" non viene valutato

Domanda 9 Si consideri lo pseudo-codice di Figura 1. Qual'è il valore di ritorno di `pluto()` se i parametri sono passati *per valore*?

- ☐ Nessuna delle altre risposte
- ☐ 1
- ☐ 3
- ☐ 0
- ☐ Dipende dal tipo di scope (statico o dinamico) utilizzato

Domanda 10 Funzione implementata dallo pseudo-codice di Figura 2:

- ☐ Non usa ricorsione in coda
- ☐ Non può essere implementata per via iterativa
- ☐ Nessuna delle altre risposte
- ☐ Causa sempre ricorsione infinita
- ☐ Usa ricorsione in coda



```
int b = 666;

int pippo(int x)
{
    x = 666; b = 1;
    return x / 2;
}

int pluto(void)
{
    int a, c;

    a = b / 333;
    c = pippo(a);

    return a + c;
}
```

Figure 1: Esempio di pseudocodice

```
int somma(int a, int b)
{
    if (a == 0) return b;
    return somma(a - 1, b + 1);
}
```

Figure 2: Esempio di pseudocodice

Domanda 11 Dato il frammento di programma (espresso in pseudo-codice) della Figura 3, quanto vale la variabile globale `c` dopo aver eseguito `topolino()`, assumendo scope dinamico?

- ☐ Non è possibile dirlo
- ☐ 6
- ☐ 3
- ☐ 14
- ☐ Nessuna delle altre risposte

Domanda 12 Si può dire che una macchina astratta che capisce il linguaggio Java non sia implementata in modo puramente compilativo perché:

- ☐ Non esistendo un vero e proprio *runtime* per Java, non si può parlare di compilazione pura
- ☐ La macchina virtuale di Java (JVM) deve comunque essere compilata
- ☐ Una macchina astratta che capisca un linguaggio di alto livello come Java non è mai implementabile con un compilatore
- ☐ Non esistono compilatori Java
- ☐ Nessuna delle altre risposte



```
int a, b, c;

void pippo(void)
{
    int a;

    a = 6;
    b = 5;
}

void pluto(void)
{
    int c;
    int b;

    pippo();
    c = 3;
    a = 4;
}

void topolino(void)
{
    int a;

    a = 1;
    b = 10;
    pluto();

    c = a + b;
}
```

Figure 3: Esempio di pseudocodice



```
int mistero(int a, int b)
{
    if (b == 0) return a;
    return mistero(a / 2, b - 1);
}
```

Figure 1: Esempio di pseudocodice

Rispondere alle domande a risposta multipla annerendo la casella corrispondente alla risposta corretta. Ogni domanda ha una ed una sola risposta corretta.

Cognome e Nome:

Matricola:

Domanda 1 La funzione implementata dallo pseudo-codice di Figura 1:

- ☐ Può causare una crescita incontrollata dello stack
- ☐ Causa sempre ricorsione infinita
- ☐ Nessuna delle altre risposte
- ☐ Non può essere implementata per via iterativa
- ☐ Non usa ricorsione in coda

Domanda 2 β -riducendo $((\lambda a.aaa)(\lambda b.b))(\lambda c.c)$ si ottiene:

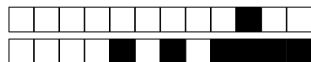
- ☐ $\lambda x.xa$
- ☐ La riduzione non termina
- ☐ aaa
- ☐ $\lambda c.c$
- ☐ Nessuna delle altre risposte

Domanda 3 Dato il frammento di programma (espresso in pseudo-codice) contenuto in Figura 2, qual'è il valore di ritorno di `topolino()`, assumendo scope statico?

- ☐ Nessuna delle altre risposte
- ☐ Non è possibile dirlo
- ☐ 0
- ☐ -3
- ☐ 11

Domanda 4 β -riducendo $(\lambda n.\lambda m.\lambda f.\lambda x.(nf)((mf)x))(\lambda f.\lambda x.fx)(\lambda f.\lambda x.x)$ si ottiene:

- ☐ $\lambda f.\lambda x.f(f(f(fx)))$
- ☐ $\lambda f.\lambda x.fx$
- ☐ x
- ☐ Nessuna delle altre risposte
- ☐ fx



```
int x, y;

void pippo(void)
{
    x = 8;
    y = 4;
}

void pluto(void)
{
    int y;

    pippo();
    y = 3;
}

int topolino(void)
{
    int x, z;

    x = 5;
    y = 15;
    z = x + y;
    pluto();

    return z - y - x;
}
```

Figure 2: Esempio di pseudocodice

Domanda 5 I record di attivazione:

- ☐ Sono necessari solo in presenza di funzioni di ordine superiore
- ☐ Sono allocati dinamicamente solo in caso di scope dinamico
- ☐ Nessuna delle altre risposte
- ☐ Devono essere esplicitamente allocati e deallocati dal codice del programma che li usa
- ☐ Sono allocati solo nello heap

Domanda 6 Dato il frammento di programma (espresso in pseudo-codice) contenuto in Figura 3, qual'è il valore di ritorno di `f1()`, assumendo scope dinamico?

- ☐ Nessuna delle altre risposte
- ☐ -1
- ☐ -5
- ☐ 5
- ☐ Non è possibile dirlo

Domanda 7 Si consideri lo pseudo-codice di Figura 4. Qual'è il valore di ritorno di `pluto()` se i parametri sono passati *per valore*?

- ☐ Nessuna delle altre risposte
- ☐ Dipende dal tipo di scope (statico o dinamico) utilizzato
- ☐ Non è possibile passare `c + 1` per valore
- ☐ 6
- ☐ 10



```
int x, y, z;

void f3(void)
{
    x = 0;
    y = 5;
}

void f2(void)
{
    int y;

    f3();
    y = 0;
    z = 10;
}

int f1(void)
{
    int x;

    x = -5;
    y = 10;
    z = x + y;
    f2();

    return z - y - x;
}
```

Figure 3: Esempio di pseudocodice

```
int c = 2;

int pippo(int a)
{
    c = c + 2;
    return a * 2;
}

int pluto(void)
{
    return(pippo(c + 1));
}
```

Figure 4: Esempio di pseudocodice



```
int c = 2;

int pippo(int a)
{
    c = c + 2;
    return a * 2;
}

int pluto(void)
{
    return(pippo(c + 1));
}
```

Figure 5: Esempio di pseudocodice

Domanda 8 Si consideri lo pseudo-codice di Figura 5. Qual'è il valore di ritorno di `pluto()` se i parametri sono passati *per nome*?

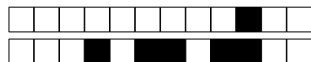
- ☐ 10
- ☐ Nessuna delle altre risposte
- ☐ 6
- ☐ 4
- ☐ Dipende dal tipo di scope (statico o dinamico) utilizzato

Domanda 9 La memoria gestita staticamente:

- ☐ E' allocata dal compilatore prima dell'esecuzione del programma. Le entità allocate staticamente in memoria risiedono in una zona fissa di memoria durante tutta l'esecuzione del programma
- ☐ Nessuna delle altre risposte
- ☐ E' allocata esplicitamente dal programma a tempo di esecuzione, ma una volta allocata è staticamente legata al programma e non può essere liberata fino alla sua terminazione
- ☐ E' una memoria a sola lettura
- ☐ E' allocata prima dell'esecuzione del programma. Le entità allocate staticamente possono essere deallocate durante l'esecuzione del programma, per liberare memoria

Domanda 10 Si può dire che una macchina astratta che capisce il linguaggio C non sia implementata in modo puramente compilativo perché:

- ☐ Gli eseguibili generati da un compilatore C in genere non eseguono direttamente sulla macchina hardware, ma su una macchina astratta che include il *runtime* del linguaggio e le funzionalità del Sistema Operativo
- ☐ Gli eseguibili generati dal compilatore vengono comunque interpretati da una macchina virtuale
- ☐ Una macchina astratta che capisca un linguaggio di alto livello come il C non è mai implementabile con un compilatore
- ☐ Il *runtime* del linguaggio C è comunque sempre interpretato
- ☐ Nessuna delle altre risposte

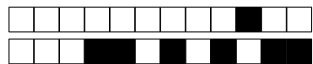


Domanda 11 Se gli array sono memorizzati *per colonne* e `char a[100][100]` è un array multidimensionale di caratteri con `a[0][0]` che ha indirizzo `0x1100`, qual'è l'indirizzo di `a[5][10]`?:

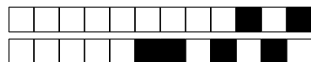
- ☐ `0x21FE`
- ☐ Nessuna delle altre risposte
- ☐ `0x22FE`
- ☐ `0x24ED`
- ☐ `0x14ED`

Domanda 12 Il costrutto `for` dei linguaggi C, C++ e Java:

- ☐ Non è un costrutto di iterazione determinata
- ☐ E' necessario a tali linguaggi per implementare qualsiasi tipo di algoritmo
- ☐ Nessuna delle altre risposte
- ☐ E' un costrutto di iterazione determinata
- ☐ Permette di sapere in anticipo quante volte il ciclo verrà ripetuto (indipendentemente dal corpo del ciclo)



+4/6/43+



Rispondere alle domande a risposta multipla annerendo la casella corrispondente alla risposta corretta. Ogni domanda ha una ed una sola risposta corretta.

Cognome e Nome:

Matricola:

Domanda 1 Un'entità esprimibile è:

- ☐ Nessuna delle altre risposte
- ☐ Un'entità che può essere generata come risultato da un'espressione complessa o da una funzione
- ☐ Un'entità che ancora non compare nell'ambiente
- ☐ Un'entità che può essere memorizzata
- ☐ Una generica entità a cui può essere dato un nome

Domanda 2 In assenza di ambiente non locale:

- ☐ Per implementare funzioni definite ricorsivamente è necessario utilizzare un fixed point combinator
- ☐ Non si possono implementare algoritmi ricorsivi
- ☐ Nessuna delle altre risposte
- ☐ Non si possono implementare algoritmi iterativi o ricorsivi
- ☐ Non si possono implementare algoritmi iterativi

Domanda 3 Dato il frammento di programma (espresso in pseudo-codice) contenuto in Figura 1, qual'è il valore di ritorno di `f1()`, assumendo scope statico?

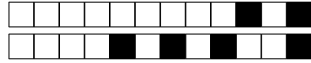
- ☐ Nessuna delle altre risposte
- ☐ Non è possibile dirlo
- ☐ 5
- ☐ -5
- ☐ 0

Domanda 4 β -riducendo $(\lambda a.((a\lambda b.\lambda c.c)\lambda d.\lambda e.d))(\lambda f.\lambda g.f)$ si ottiene:

- ☐ La riduzione non termina
- ☐ $\lambda b.\lambda c.b$
- ☐ c
- ☐ $\lambda b.\lambda c.c$
- ☐ Nessuna delle altre risposte

Domanda 5 β -riducendo $(\lambda a.aaa)((\lambda b.b)(\lambda c.c))$ si ottiene:

- ☐ La riduzione non termina
- ☐ $\lambda x.xa$
- ☐ Nessuna delle altre risposte
- ☐ $\lambda a.a$
- ☐ aaa



```
int x, y, z;

void f3(void)
{
    x = 0;
    y = 5;
}

void f2(void)
{
    int y;

    f3();
    y = 0;
    z = 10;
}

int f1(void)
{
    int x;

    x = -5;
    y = 10;
    z = x + y;
    f2();

    return z - y - x;
}
```

Figure 1: Esempio di pseudocodice

Domanda 6 Si può dire che una macchina astratta che capisce il linguaggio C non sia implementata in modo puramente compilativo perché:

- ☐ Gli eseguibili generati da un compilatore C in genere non eseguono direttamente sulla macchina hardware, ma su una macchina astratta che include il *runtime* del linguaggio e le funzionalità del Sistema Operativo
- ☐ Il *runtime* del linguaggio C è comunque sempre interpretato
- ☐ Una macchina astratta che capisca un linguaggio di alto livello come il C non è mai implementabile con un compilatore
- ☐ Nessuna delle altre risposte
- ☐ Gli eseguibili generati dal compilatore vengono comunque interpretati da una macchina virtuale

Domanda 7 In presenza di variabili modificabili:

- ☐ Nessuna delle altre risposte
- ☐ Esistono un *Ambiente* che associa valori denotabili (fra cui le locazioni di memoria) a nomi ed una *Memoria* che associa locazioni di memoria a valori memorizzabili
- ☐ Non esistono valori denotabili
- ☐ La valutazione del comando di assegnamento restituisce sempre un valore
- ☐ Il comando di assegnamento non ha effetti collaterali



```
int c = 2;

int pippo(int a)
{
    c = c + 2;
    return a * 2;
}

int pluto(void)
{
    return(pippo(c + 1));
}
```

Figure 2: Esempio di pseudocodice

```
int mistero(int a, int b)
{
    if (b == 0) return a;
    return mistero(a / 2, b - 1);
}
```

Figure 3: Esempio di pseudocodice

Domanda 8 Si consideri lo pseudo-codice di Figura 2. Qual'è il valore di ritorno di `pluto()` se i parametri sono passati *per valore*?

- ☐ 10
- ☐ Non è possibile passare `c + 1` per valore
- ☐ Dipende dal tipo di scope (statico o dinamico) utilizzato
- ☐ 6
- ☐ Nessuna delle altre risposte

Domanda 9 L'allocazione dinamica della memoria:

- ☐ E' sempre effettuata solo dal compilatore o dall'interprete
- ☐ Può essere fatta solo dallo *heap*
- ☐ Può essere fatta solo dallo *stack*
- ☐ Nessuna delle altre risposte
- ☐ Può essere fatta sia dallo *stack* che dallo *heap*

Domanda 10 La funzione implementata dallo pseudo-codice di Figura 3:

- ☐ Può causare una crescita incontrollata dello stack
- ☐ Non usa ricorsione in coda
- ☐ Nessuna delle altre risposte
- ☐ Causa sempre ricorsione infinita
- ☐ Non può essere implementata per via iterativa

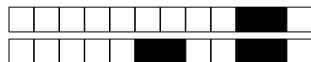


Domanda 11 La *frammentazione interna* causa:

- ☐ L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente
- ☐ Un rallentamento rilevante nelle operazioni di allocazione della memoria
- ☐ Nessuna delle altre risposte
- ☐ Uno spreco di memoria
- ☐ Il funzionamento non corretto di programmi che allocano memoria dinamicamente

Domanda 12 Se gli array sono memorizzati *per colonne* ed `int a[100][100]` è un array multidimensionale di interi (si assuma che la dimensione di un intero sia 4 byte) con `a[0][0]` che ha indirizzo `0x5000`, qual'è l'indirizzo di `a[5][10]`?:

- ☐ `0x5510`
- ☐ `0x53ED`
- ☐ Nessuna delle altre risposte
- ☐ `0x500F`
- ☐ `0x41FE`



Rispondere alle domande a risposta multipla annerendo la casella corrispondente alla risposta corretta. Ogni domanda ha una ed una sola risposta corretta.

Cognome e Nome:

Matricola:

Domanda 1 Il concetto di *variabile modificabile*:

- ☐ E' l'unico concetto utilizzabile quando si parla di variabili
- ☐ E' imposto dall'architettura di Von Neumann (variabili non modificabili richiederebbero macchine astratte caratterizzate da memoria a sola lettura)
- ☐ Nessuna delle altre risposte
- ☐ E' tipico del paradigma di programmazione imperativo
- ☐ Permette di evitare il fenomeno dell'*aliasing*

Domanda 2 La memoria gestita staticamente:

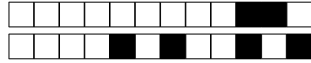
- ☐ Nessuna delle altre risposte
- ☐ E' allocata esplicitamente dal programma a tempo di esecuzione, ma una volta allocata è staticamente legata al programma e non può essere liberata fino alla sua terminazione
- ☐ E' una memoria a sola lettura
- ☐ E' allocata dal compilatore prima dell'esecuzione del programma. Le entità allocate staticamente in memoria risiedono in una zona fissa di memoria durante tutta l'esecuzione del programma
- ☐ E' allocata prima dell'esecuzione del programma. Le entità allocate staticamente possono essere deallocate durante l'esecuzione del programma, per liberare memoria

Domanda 3 Dato il frammento di programma (espresso in pseudo-codice) della Figura 1, quanto vale la variabile globale `c` dopo aver eseguito `topolino()`, assumendo scope dinamico?

- ☐ Nessuna delle altre risposte
- ☐ Non è possibile dirlo
- ☐ 6
- ☐ 3
- ☐ 14

Domanda 4 Si consideri lo pseudo-codice di Figura 2. Qual'è il valore di ritorno di `f(1,r(1),1)` se i parametri sono passati *per valore*?

- ☐ Nessuna delle altre risposte
- ☐ Si ha ricorsione infinita
- ☐ 1
- ☐ Non è possibile dirlo senza conoscere il tipo di scope (statico o dinamico) utilizzato
- ☐ 0



```
int a, b, c;

void pippo(void)
{
    int a;

    a = 6;
    b = 5;
}

void pluto(void)
{
    int c;
    int b;

    pippo();
    c = 3;
    a = 4;
}

void topolino(void)
{
    int a;

    a = 1;
    b = 10;
    pluto();

    c = a + b;
}
```

Figure 1: Esempio di pseudocodice

```
int r(int x)
{
    return r(x - 1);
}

int f(int a, int b, int c)
{
    if (c == 1) return a; else return b;
}
```

Figure 2: Esempio di pseudocodice



```
int x, y, z;

void minni(void)
{
    x = 4;
    y = 8;
}

void paperino(void)
{
    int y;

    minni();
    y = 1;
    z = 666;
}

int topolino(void)
{
    int x;

    x = 5;
    y = 15;
    z = x + y;
    paperino();

    return z - y - x;
}
```

Figure 3: Esempio di pseudocodice

Domanda 5 Se gli array sono memorizzati *per righe* ed `int a[100][100]` è un array multidimensionale di interi (si assuma che la dimensione di un intero sia 4 byte) con `a[0][0]` che ha indirizzo `0x5000`, qual'è l'indirizzo di `a[5][10]`?:

- ☐ `0x5510`
- ☐ Nessuna delle altre risposte
- ☐ `0x51FE`
- ☐ `0x53ED`
- ☐ `0x57F8`

Domanda 6 Dato il frammento di programma (espresso in pseudo-codice) contenuto in Figura 3, qual'è il valore di ritorno di `topolino()`, assumendo scope statico?

- ☐ -3
- ☐ 0
- ☐ 14
- ☐ Non è possibile dirlo
- ☐ Nessuna delle altre risposte



```
int c = 2;

int pippo(int a)
{
    c = c + 2;
    return a * 2;
}

int pluto(void)
{
    return(pippo(c + 1));
}
```

Figure 4: Esempio di pseudocodice

Domanda 7 Si consideri lo pseudo-codice di Figura 4. Qual'è il valore di ritorno di `pluto()` se i parametri sono passati *per riferimento*?

- ☐ Dipende dal tipo di scope (statico o dinamico) utilizzato
- ☐ 10
- ☐ Non è possibile passare `c + 1` per riferimento
- ☐ Nessuna delle altre risposte
- ☐ 6

Domanda 8 β -riducendo $(\lambda x.xy)(\lambda z.zx)(\lambda z.zx)$ si ottiene:

- ☐ $yx(\lambda z.zx)$
- ☐ $(\lambda x.xy)yx$
- ☐ Nessuna delle altre risposte
- ☐ xyz
- ☐ La riduzione non termina

Domanda 9 β -riducendo $(\lambda n.\lambda f.\lambda x.f((nf)x))(\lambda f.\lambda x.f(f(f(fx))))$ si ottiene:

- ☐ La riduzione non termina
- ☐ fx
- ☐ $\lambda f.\lambda x.f f f f f x$
- ☐ Nessuna delle altre risposte
- ☐ $\lambda f.\lambda x.f(f(f(fx)))$

Domanda 10 Un *compilatore* da un linguaggio $\mathcal{L}$ ad un linguaggio $\mathcal{L}_O$ è:

- ☐ L'implementazione di una macchina astratta scritta nel linguaggio $\mathcal{L}_O$, che capisce programmi scritti nel linguaggio $\mathcal{L}$
- ☐ Una implementazione di macchine astratte indipendente dalla macchina fisica
- ☐ Un programma che trasforma un programma $P^{\mathcal{L}}$ (espresso nel linguaggio $\mathcal{L}$) in un programma $P^{\mathcal{L}_O}$ (espresso nel linguaggio $\mathcal{L}_O$) tale che per ogni input I si ha $P^{\mathcal{L}}(I) = P^{\mathcal{L}_O}(I)$
- ☐ Nessuna delle altre risposte
- ☐ Un programma scritto nel linguaggio $\mathcal{L}_O$ che riceve come ingresso un programma $P^{\mathcal{L}}$ (espresso nel linguaggio $\mathcal{L}$) ed il suo input I generando lo stesso output che genera $P^{\mathcal{L}}$ con input I



Domanda 11 In presenza di variabili modificabili:

- ☐ Nessuna delle altre risposte
- ☐ Non esistono valori denotabili
- ☐ Esistono un *Ambiente* che associa valori denotabili (fra cui le locazioni di memoria) a nomi ed una *Memoria* che associa locazioni di memoria a valori memorizzabili
- ☐ Il comando di assegnamento non ha effetti collaterali
- ☐ La valutazione del comando di assegnamento restituisce sempre un valore

Domanda 12 L'allocazione dinamica della memoria:

- ☐ Può essere fatta sia dallo *stack* che dallo *heap*
- ☐ Nessuna delle altre risposte
- ☐ E' sempre effettuata solo dal compilatore o dall'interprete
- ☐ Può essere fatta solo dallo *stack*
- ☐ Può essere fatta solo dallo *heap*

